# Code Obfuscation with Tigress

---

*Advanced Security*

Master Cybersecurity — UGA

**DANG Nhat Minh**

Tool: Tigress 4.2.1

November 12, 2026

I used Tigress to obfuscate two small C programs and then read the generated code to point out the main obfuscation technique of each transformation. Every transformation was recompiled and run: the output is always the same as the original, so the behaviour is preserved. The general command is `tigress --Transform=T --Functions=f in.c --out=result.c`.

**Source code, generated files and gdb transcripts:**
`https://github.com/sunnylucy1010/M2CYSEC/tree/main/AdvancedSecurity/tp2-code-obfuscation`

## 1 Virtualize (a virtual machine)

**Original program** `simple.c` (returns 42):

```
int main(){ int x = 42; return x++; }
```

After `--Transform=Virtualize`, the 47-byte file becomes about 44 KB but still returns 42. Tigress replaced the code by a small **virtual machine**: my program is now stored as **bytecode** (an array of bytes) and executed by an **interpreter loop (dispatcher)** that reads one opcode at a time:

```
unsigned char $array[1][53];        // bytecode = my program encoded as bytes
unsigned char *$pc;                 // virtual program counter (points in $array)
union _node   *$sp;                 // virtual stack pointer
while (1) {
  switch (*($pc[0])) {              // fetch an opcode, then execute it
    case constant: ... ;            // push a value
    case store:    ... ;            // write a variable
    case PlusA:    ($sp-1)→_int = ($sp)→_int + ($sp-1)→_int; $sp--;  // <-- example opcode:
    ADD
    case return:   return ($sp)→_int;
  }
}
```

**Main feature:** the logic is no longer visible as instructions; it is data (bytecode) interpreted by a VM. **Example opcode:** `PlusA` is an addition done on the virtual stack.

**Checking with gdb.** Breaking on the `switch` and printing the opcode each iteration gives this real output:

```
(gdb) break at the switch; print PC and opcode each loop:
iter1  PC=0x555555558060  opcode=0x7e
iter2  PC=0x555555558065  opcode=0x26
iter3  PC=0x55555555806a  opcode=0xaf
iter4  PC=0x55555555806b  opcode=0x26
```

Here `0x7e=constant` (push 42), `0x26=local` (push &x), `0xaf=store` (x=42): exactly the original `x=42; return x++;`. The PC moves forward through `$array` and the loop runs once per virtual instruction, confirming that `main` is now an interpreter. Changing `--VirtualizeDispatch` (switch / direct / indirect / call) only changes how the next opcode is reached (a `switch`, a computed `goto`, or function pointers); the bytecode stays the same.

## 2 Flatten and Opaque Predicates

**Original program** `collatz.c` — function `classify(n)` with a loop and some `if` (returns 211 for n=27):

```
int classify(int n){
  int steps = 0;
  if (n ≤ 0) return -1;
  while (n ≠ 1){ if (n%2==0) n = n/2; else n = 3*n+1; steps++; }
  if (steps > 10) return steps + 100;
  return steps;
}
```

**Flatten.** The `if/while` structure disappears: every block becomes a `case` and a variable `NEXT` says which block runs next.

```
int classify(int n){
  unsigned long NEXT = 9;
  while (1){ switch (NEXT){
    case 9:  steps = 0;            NEXT = 10; break;   // entry
    case 10: if (n≤0) NEXT=12; else NEXT=3; break;    // if (n≤0)
    case 7:  if (n%2==0) NEXT=5; else NEXT=1; break;   // if inside the loop
    case 1:  n = 3*n + 1;         NEXT = 0;  break;
    case 12: return -1;  /* ... other cases ... */
  }}
}
```

**Main feature:** the control-flow graph (loop + branches) is turned into a flat state machine, so the original structure is hidden. **Checking with gdb** (a watchpoint on the state variable, n=27) shows the blocks being selected one after another — real output:

```
(gdb) break classify
(gdb) run 27
(gdb) watch _TIG_FN_Krvf_1_classify_next
   Old value = 93824992247256
   New value = 9
   Old value = 9
   New value = 10
   Old value = 10
   New value = 3
   Old value = 3
   New value = 7
   Old value = 7
   New value = 1
   Old value = 1
   New value = 0
```

The value goes `9 -> 10 -> 3 -> 7 -> 1 -> 0`: the state machine running through the blocks.

**AddOpaque.** Tigress adds fake branches guarded by an **opaque predicate** — a condition that is always true but hard to prove statically. It first builds a small data structure at run time, then uses it inside a huge always-true condition:

```
cond = (int)( ...(p == (node*)((unsigned long)(q ^ K) ^ K))... );  // 11,666 chars, always == 1
if (cond) { steps = 0; } else { /* dead code */ }
```

**Main feature:** real code is wrapped in conditions that always hold, plus dead branches, which adds noise and makes the program look much more complex than it is. **Checking with gdb**, a watchpoint on the guard shows it really is constant — real output:

```
(gdb) break classify
(gdb) run 27
(gdb) watch OPAQUE7
   Old value = 32767
   New value = 1
```

`OPAQUE7` becomes `1`, so the branch is always taken: it is an opaque (always-true) predicate.

## 3 Data Transformations and Self-Modification

**EncodeArithmetic.** Normal operations are rewritten as equivalent but strange bit operations:

| Original | Obfuscated |
|---|---|
| `n = 3*n + 1` | `n = ((3*n \| 1) « 1) - (3*n ^ 1)` |
| `steps + 100` | `(steps - ~100) - 1` |

**Main feature:** the arithmetic is hidden; `x+1` no longer looks like an addition.

**EncodeLiterals.** The constant `42` does not appear anymore in the binary; it is recomputed at run time, so a simple search for the value fails.

**SelfModify.** The program **modifies its own machine code at run time** (same idea as Part 1), so what a static tool reads is not what really executes. To let it run, the code section must be writable; Tigress ships `tigress_post --Action=writable` for this (otherwise the program gets a SIGSEGV because modern systems map `.text` read-only — W^X — exactly the problem Part 1 solved with `mprotect`). **gdb proves the self-modification**: stopping on the write instruction and reading the target bytes before and after shows the code change itself:

```
(gdb) break *(classify+5392)    # stop at the self-modifying write: mov %rdx,(%rax)
(gdb) run 27
(gdb) x/4xb $rax                # target bytes BEFORE the write (= real code of classify)
   <classify+5494>:  0xf1  0x7e  0x05  0x4d
(gdb) stepi                     # execute the single write instruction
(gdb) x/4xb $rax                # SAME address AFTER: the code bytes changed
   <classify+5494>:  0x34  0xd8  0xff  0xff
```

The instruction at `classify+5392` writes into `classify+5494` — both inside `.text` — and the 4 bytes there change from `f1 7e 05 4d` to `34 d8 ff ff` after a single step: the function rewrites its own instructions while running.

## 4 Summary

| Transformation | Main feature | Makes reversing harder because |
|---|---|---|
| Virtualize | bytecode + VM interpreter | you see a VM, not the real code |
| Flatten | flat state machine (`NEXT`) | the loop/branch structure is gone |
| AddOpaque | always-true conditions + dead code | fake complexity, hard to prune |
| EncodeArithmetic | operations as bit tricks | data flow is hidden |
| EncodeLiterals | constants recomputed | constants are not in the binary |
| SelfModify | code rewrites itself at run time | static code ≠ executed code |